SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)

Course Code:	CSA65	**Course Title:**	Generative AI and Large Scale Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	**Assessment:**	Assessment Tool 3 - Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)
Weightage:	10% of CIA	**Total Marks:**	2 * 50= 100
CO4 Statement:	*Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.*(BL3, BL4,BL5)		
Student Name:	S.Sirivally	**Reg. No.:**	192472371
Section / Batch:	2024	**Date:**	26-08-26

Instructions to Students

- Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.
- Develop and execute the assigned **Python program** using Text-to-Image, Speech-to-Text, or Text-to-Speech.
- Explain the **working principle and program workflow**.
- Demonstrate the program with **suitable inputs and outputs**.
- Use appropriate **libraries/APIs** and include basic **error handling**.
- Show the **execution output/screenshots**.
- For integrated tasks, demonstrate the complete **multimodal workflow**.
- Include a **workflow/architecture diagram or comparison table**.
- Mention a **real-world application** of the implemented solution.
- Explain the **strengths and limitations** of the approach.
- Include proper **references/citations** for the libraries, APIs, models, or research sources used.

Code of Conduct

I S.Sirivally / 192472371 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___Sirivally.S___

SECTION A : Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)

Text-to-Image Programming

1. Write a program that handles an empty text prompt and displays an appropriate error message.

Problem Statement

Write a Python program that accepts a text prompt and generates an image using a Text-to-Image AI model. The program must also handle an empty text prompt and display an appropriate error message.

Objective

The objective of this program is to:

- Accept a text prompt from the user.
- Validate the entered prompt.
- Display an error message when the prompt is empty.
- Send a valid prompt to a Text-to-Image AI model.
- Generate an image based on the prompt.
- Save and display the generated image.
- Handle API and runtime errors.

Technologies and Libraries Used

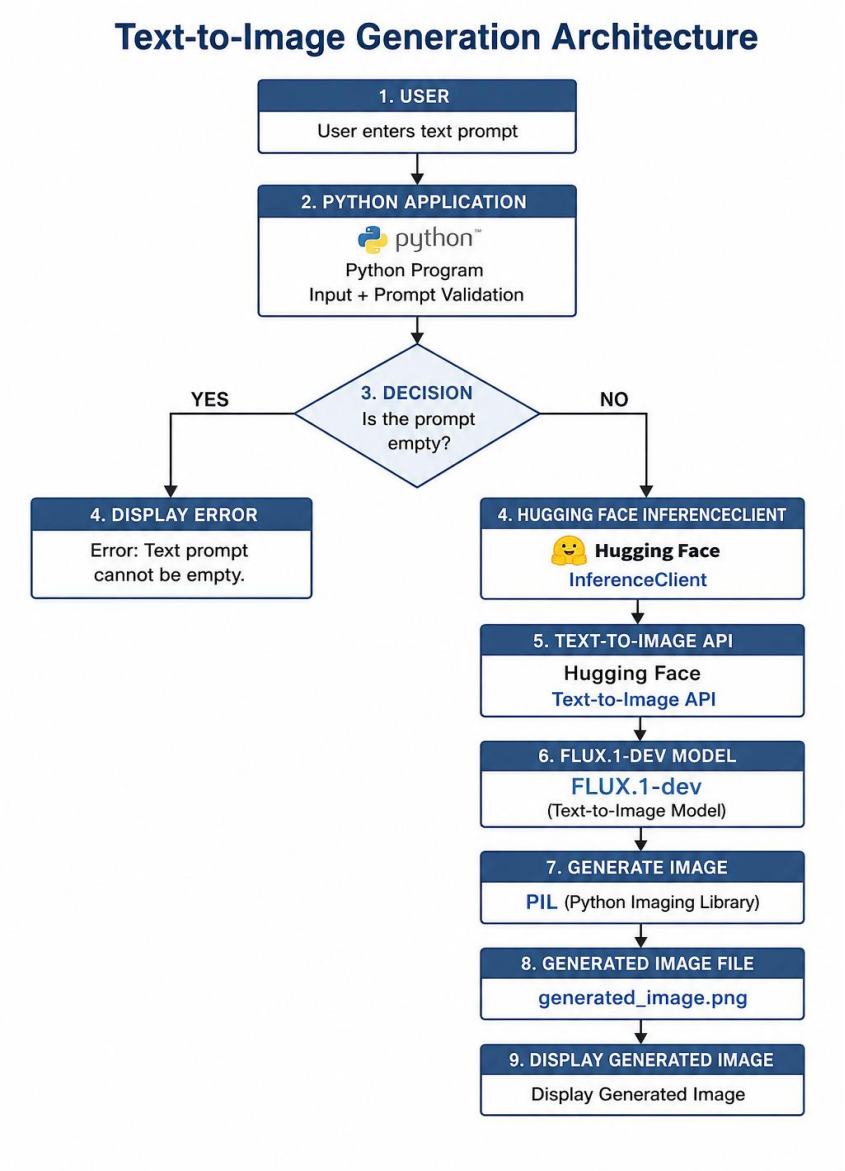
Technology / Library	Purpose
Python	Programming language
Hugging Face	AI inference platform
huggingface_hub	Connects Python program to Hugging Face inference
PIL	Saving and displaying generated images
FLUX.1-dev	Text-to-Image model

Working Principle

The program follows these steps:

1. The user enters a text prompt.
2. The program removes unnecessary spaces from the input.
3. It checks whether the prompt is empty.
4. If the prompt is empty, an error message is displayed.
5. If the prompt is valid, it is sent to the Hugging Face Text-to-Image model.
6. The AI model generates an image based on the prompt.
7. The generated image is saved as generated_image.png.
8. The image is displayed to the user.
9. Errors are handled using exception handling.

Workflow / Architecture



Python Program

```
from huggingface_hub import InferenceClient

# Hugging Face API token

HF_TOKEN = "hf_YOUR_TOKEN_HERE"

# Create Hugging Face client

client = InferenceClient(

    api_key=HF_TOKEN

)
```

```
# Take input from the user
prompt = input("Enter a text prompt: ").strip()
# Check for empty prompt
if not prompt:
    print("Error: Text prompt cannot be empty.")
else:
    try:
        print("Generating image...")
        # Generate image using FLUX.1-dev
        image = client.text_to_image(
            prompt=prompt,
            model="black-forest-labs/FLUX.1-dev"
        )
        # Save generated image
        image.save("generated_image.png")
        print("Image generated successfully!")
        print("Image saved as generated_image.png")
        # Display image
        image.show()
    except Exception as e:
        print("Error:", e)
```

Test Cases and Outputs

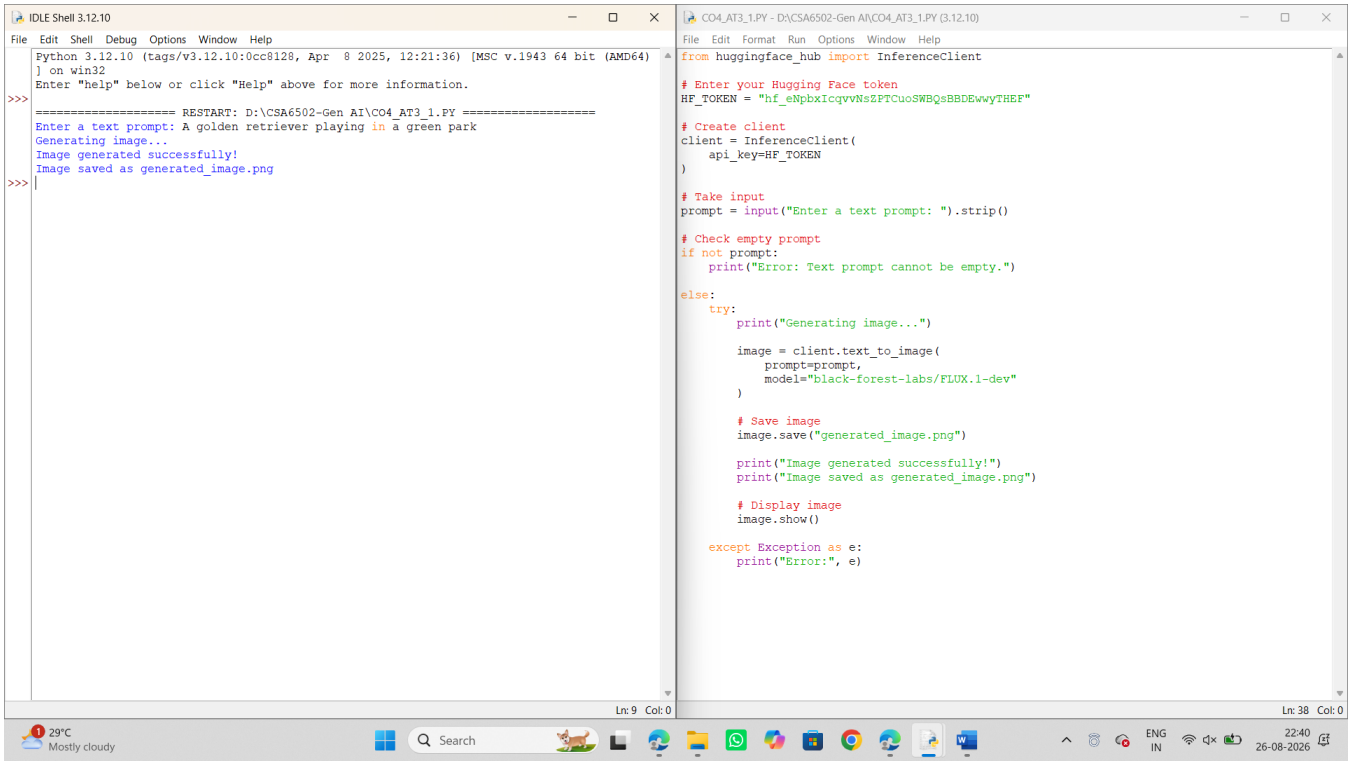
You have three test cases.

Test Case 1 – Animal Image

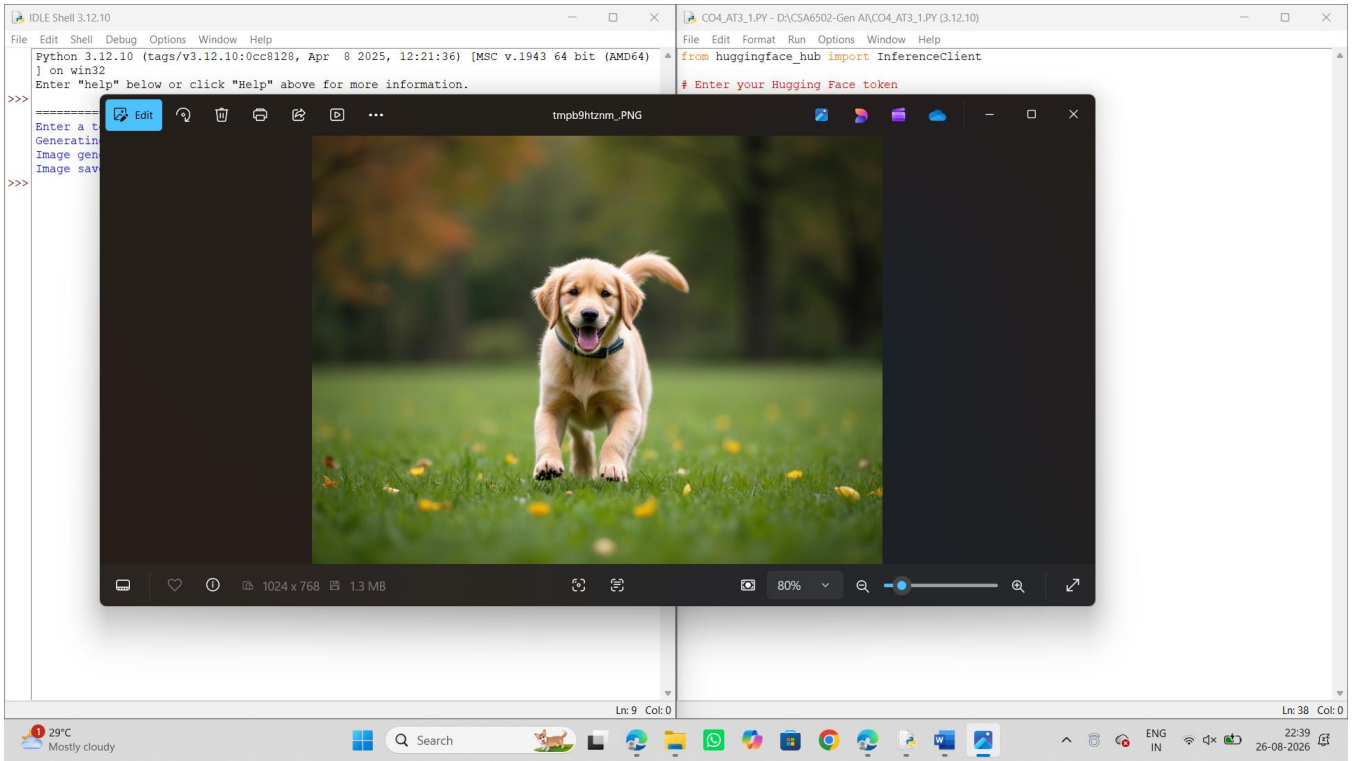
Input

A golden retriever playing in a green park

Output



Screenshot



Test Case 2 – Agriculture Image

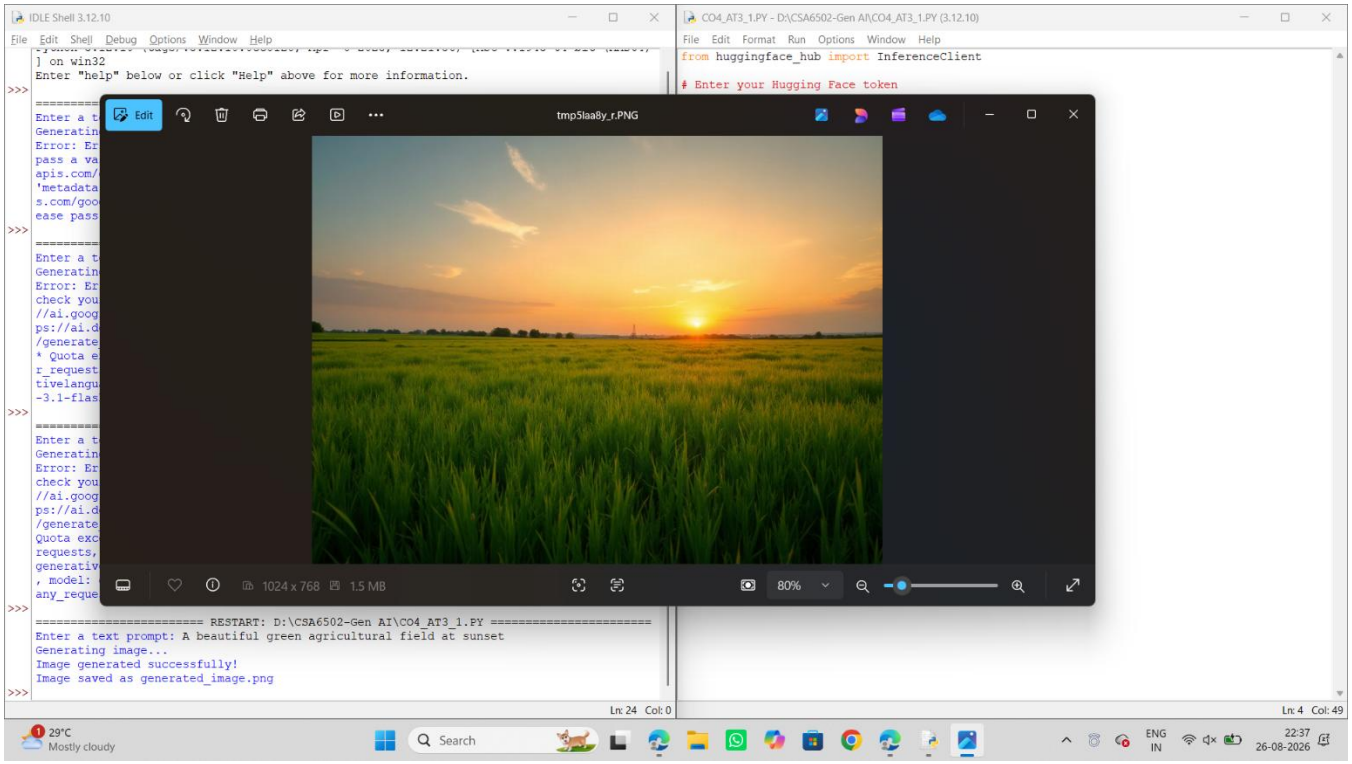
Input

A beautiful green agricultural field at sunset

Output

```
>>>
========================= RESTART: D:\CSA6502-Gen AI\CO4_AT3_1.PY =========================
Enter a text prompt: A beautiful green agricultural field at sunset
Generating image...
Image generated successfully!
Image saved as generated_image.png
>>>
```

Screenshot

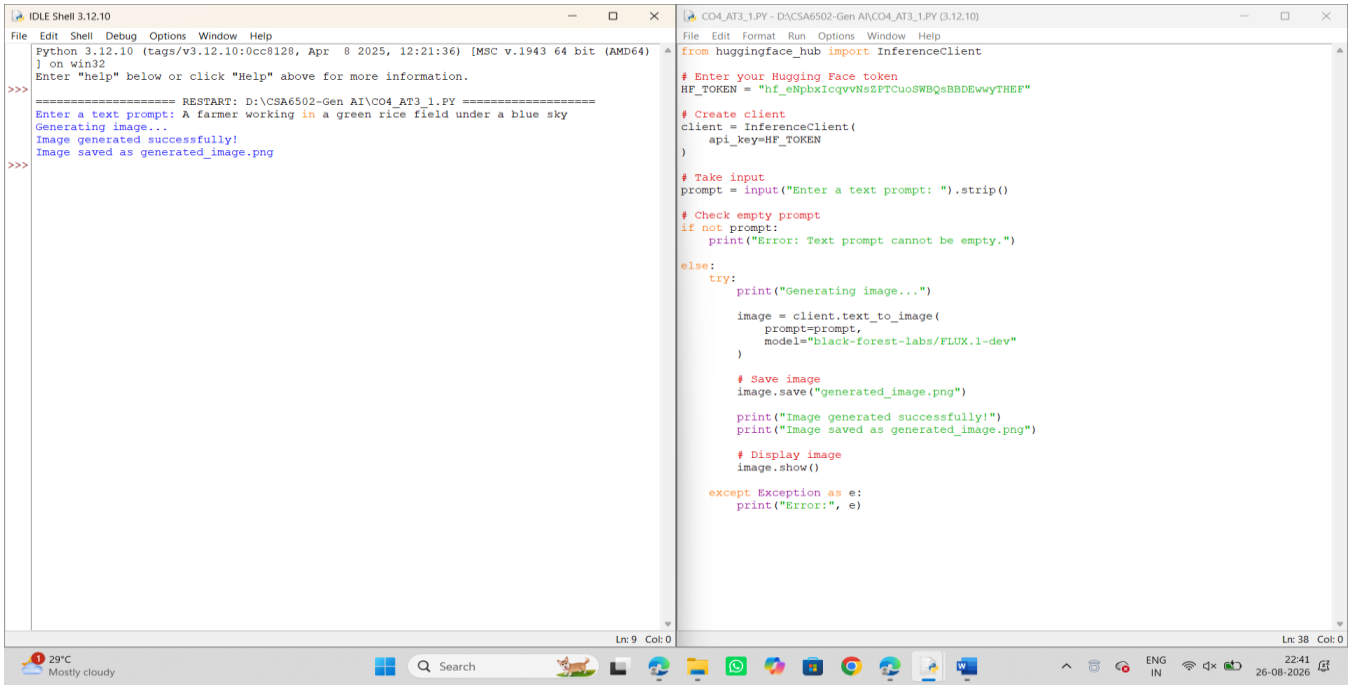


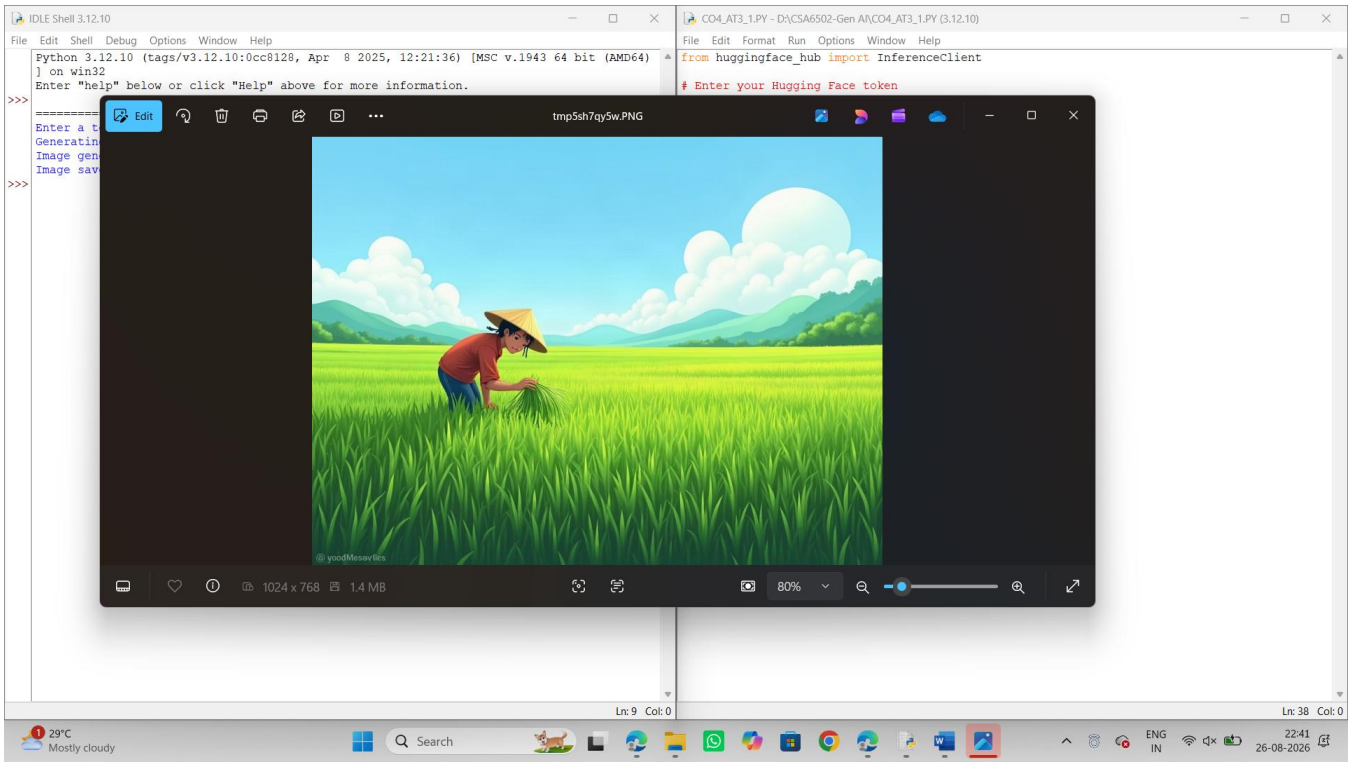
Test Case 3 –Farmer Image

Input

A farmer working in a green rice field under a blue sky

Output





Real-World Applications

Text-to-Image generation can be used in:

- Agricultural visualization.
- Advertisement and marketing.
- Graphic design.
- Educational content creation.

- Game and animation development.
- Product visualization.
- Social media content generation.

Example

In agriculture, a farmer or agricultural organization could generate visual representations of crop fields, farming techniques, irrigation systems, or sustainable farming practices from text descriptions.

Strengths

1. Generates images from simple natural-language descriptions.
2. Reduces the need for manual graphic designing.
3. Can create different types of visual content quickly.
4. Easy to integrate into Python applications.
5. Useful for prototyping and visualization.

Limitations

1. Image generation depends on API availability.
2. Internet connectivity is required.
3. API usage may have rate or quota limitations.
4. Generated images may not always exactly match the prompt.
5. Results depend on the quality and specificity of the prompt.
6. API tokens must be kept secure.

References

[1] Hugging Face, "Inference Providers – Text to Image."
This documentation describes text-to-image generation using the Hugging Face InferenceClient and explains the text_to_image() API.

Hugging Face – Text-to-Image Documentation

[2] Hugging Face, "InferenceClient API Reference."
This reference documents the Python InferenceClient, including the text_to_image() method and its parameters and return value.

Hugging Face InferenceClient Documentation

[3] Black Forest Labs / Hugging Face, "FLUX.1-dev."
The FLUX.1-dev model is used as the text-to-image model in the implementation.

FLUX.1-dev Model

2. Modify an STT program to display the recognized text on the screen.

Problem Statement

Modify a Speech-to-Text program so that it accepts speech through a microphone, converts the speech into text using a Generative AI model, and displays the recognized text on the screen.

Objective

The objective of this program is to:

- Capture speech through a microphone.
- Record the user's voice.
- Save the recorded speech as an audio file.
- Send the audio to Gemini.
- Convert speech into text.
- Display the recognized text.
- Handle speech recognition and runtime errors.

Technologies and Libraries Used

Technology / Library	Purpose
Python	Programming language
Google Gemini API	Speech/audio understanding
google-genai	Gemini API integration
SpeechRecognition	Microphone/audio capture
PyAudio	Microphone access
WAV	Audio file format

Working Principle

The program works as follows:

1. The program initializes the Gemini API client.
2. A speech recognizer is created.
3. The microphone is activated.
4. Background noise is adjusted.
5. The user speaks into the microphone.
6. The speech is captured as audio.
7. The audio is saved as a WAV file.
8. The WAV file is uploaded to Gemini.
9. Gemini processes the audio.
10. Gemini generates the speech transcription.
11. The recognized text is displayed on the screen.
12. Errors are handled using exception handling.

Workflow / Architecture

Python Program

```
import speech_recognition as sr

from google import genai

# Gemini API key

API_KEY = "YOUR_GEMINI_API_KEY"

# Create Gemini client

client = genai.Client(api_key=API_KEY)
```

```
# Create speech recognizer
recognizer = sr.Recognizer()
print("================================")
print("     GEMINI SPEECH-TO-TEXT")
print("================================")
try:
    # Take speech from microphone
    with sr.Microphone() as source:
        print("Adjusting for background noise...")
        recognizer.adjust_for_ambient_noise(
            source,
            duration=1
        )
        print("Please speak...")
        audio = recognizer.listen(source)
    # Save recorded speech as WAV
    with open("speech.wav", "wb") as file:
        file.write(audio.get_wav_data())
    print("Speech recorded successfully.")
    print("Sending audio to Gemini...")
    # Upload audio file to Gemini
    audio_file = client.files.upload(
        file="speech.wav"
    )
    # Convert speech to text
```

```
    response = client.models.generate_content(
        model="gemini-3.6-flash",
        contents=[
            audio_file,
            "Transcribe the speech in this audio accurately. "
            "Return only the recognized text."
        ]
    )
    # Display recognized text
    print("\nRecognized Text:")
    print(response.text)
except sr.WaitTimeoutError:
    print("Error: No speech detected.")
except sr.UnknownValueError:
    print("Error: Could not understand the speech.")
except Exception as e:
    print("Error:", e)
```

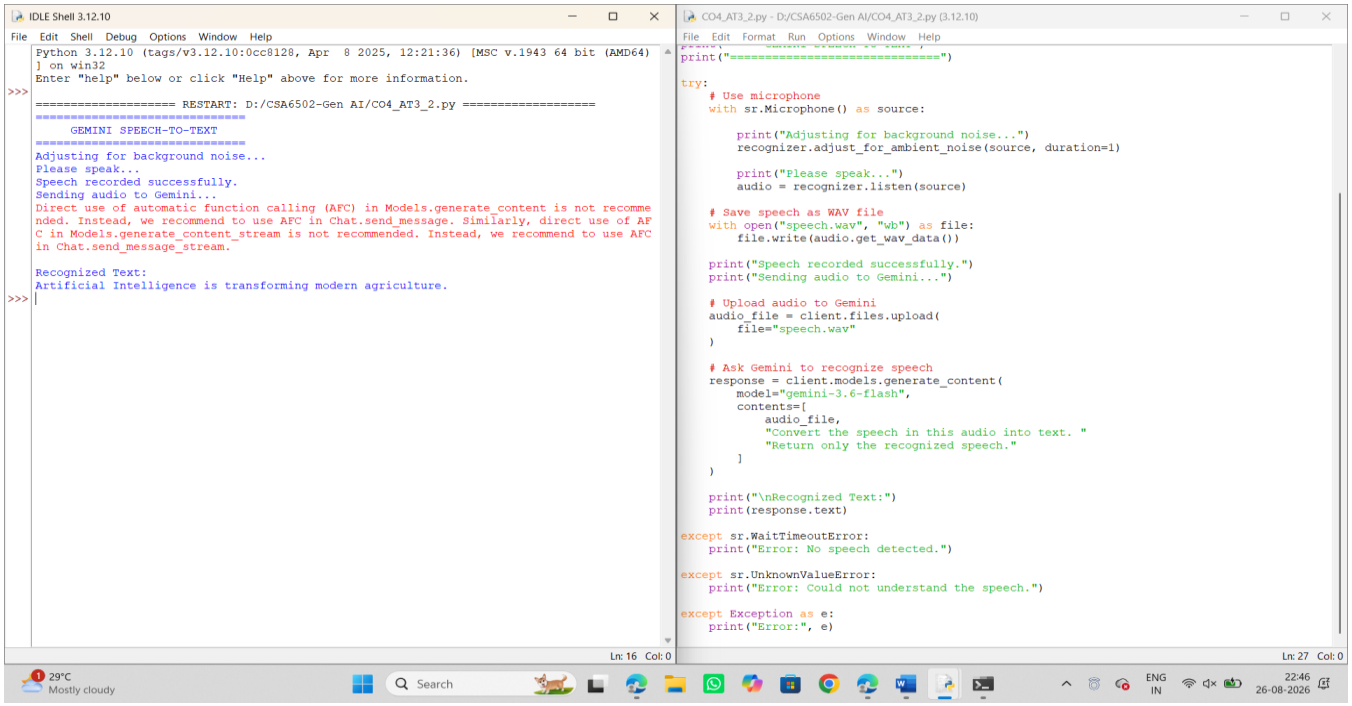
Test Cases and Outputs

Test Case 1

Speech Input

Speak: Artificial Intelligence is transforming modern agriculture.

Screenshot



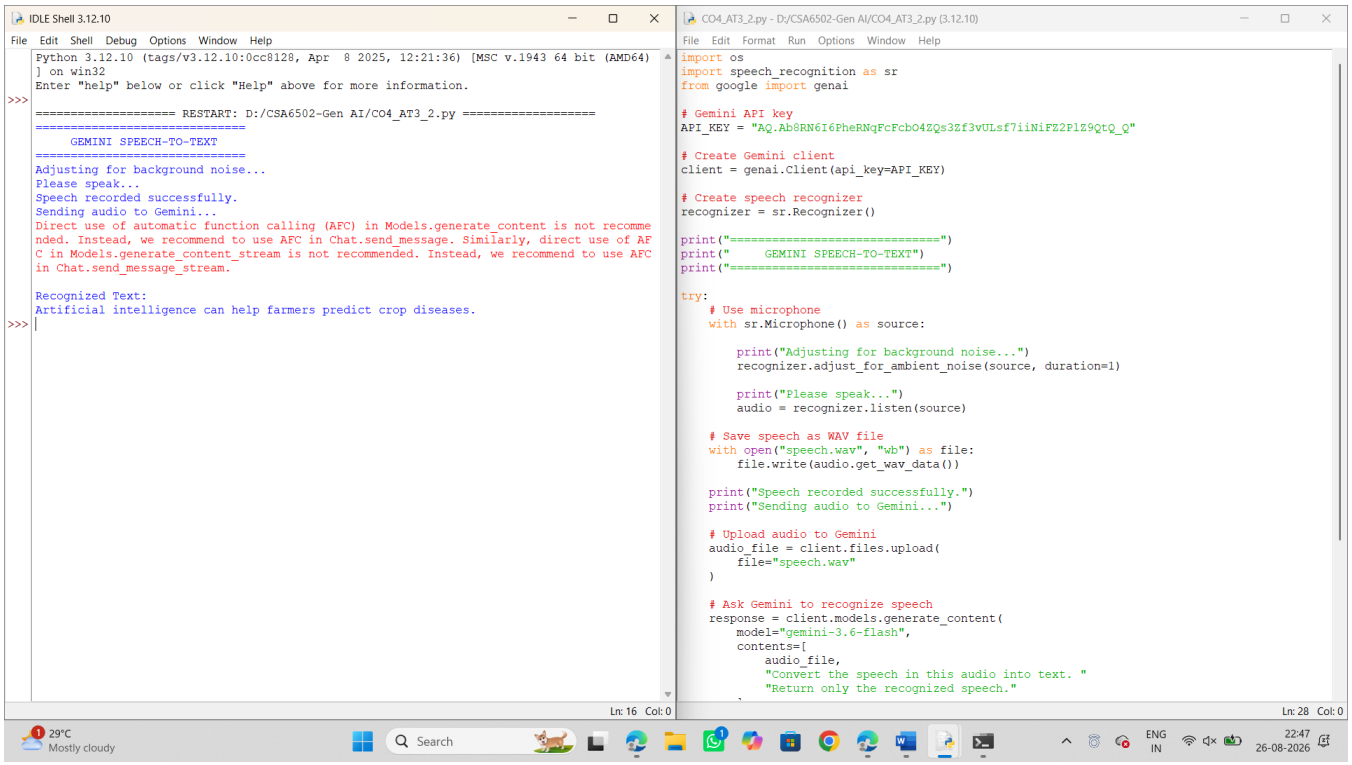
Test Case 2

Speech Input

Speak:

Artificial intelligence can help farmers predict crop diseases.

Screenshot



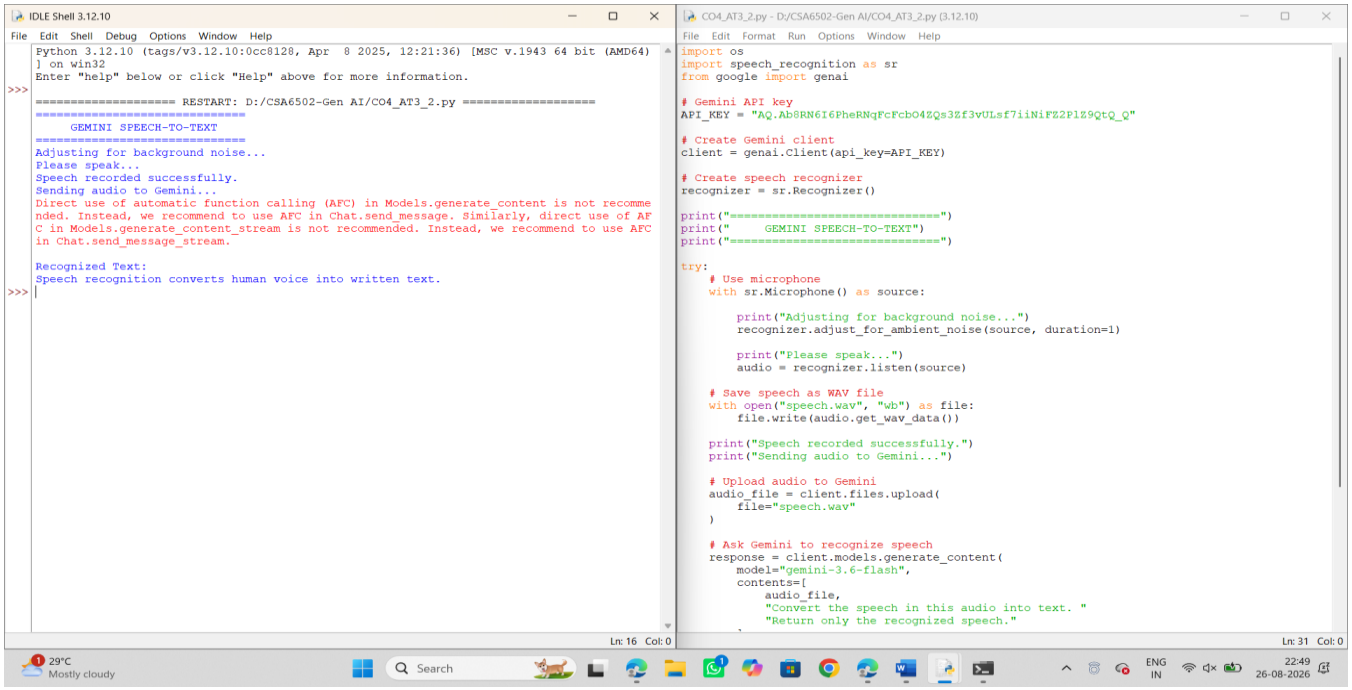
Test Case 3

Speech Input

Speak:

Speech recognition converts human voice into written text.

Screenshot



Real-World Applications

Speech-to-Text technology has many applications, including:

- Voice assistants.
- Automatic meeting transcription.
- Lecture transcription.
- Accessibility applications.
- Customer service systems.
- Healthcare documentation.
- Voice-controlled applications.
- Agricultural voice-based systems.

Example

A farmer can use a voice-based agricultural application to speak a query such as:

"What is the best fertilizer for rice crops?"

The speech can be converted into text and then processed by an AI system to provide an appropriate response.

Strengths

1. Allows natural voice-based interaction.
2. Eliminates the need to type input.
3. Useful for accessibility.
4. Can process natural human speech.
5. Gemini can understand audio context.
6. Easy to integrate into Python applications.

Limitations

1. Requires a microphone.
2. Internet connectivity is required for Gemini API processing.
3. Recognition accuracy can decrease because of background noise.
4. Different accents and pronunciations can affect transcription.
5. API usage may have quota or rate limitations.
6. API keys must be securely stored.

References

[1] Google, "Gemini API – Audio Understanding."
Google's documentation describes how Gemini accepts audio input and can generate text responses, including speech transcription.

Google Gemini API – Audio Documentation

[2] Google, "Gemini API Documentation."
This documentation provides information about using Google's Gemini API and the Python SDK.

Google Gemini API Documentation

[3] SpeechRecognition Python Library Documentation.
The SpeechRecognition library is used to access the microphone and capture speech in the Python program.

SpeechRecognition Documentation

[4] PyAudio Documentation.
PyAudio provides Python bindings for PortAudio and is used by the speech-recognition setup for microphone/audio input.

PyAudio on PyPI

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Pipeline Implementation	30%	Correctly implements a working text-to-image AND speech-to-text/text-to-speech pipeline	Both implemented with minor issues	Only one modality implemented correctly	Neither modality functions correctly
Output Quality & Relevance	25%	Generated outputs are high-quality and clearly relevant to inputs	Outputs are mostly relevant	Outputs are of inconsistent quality	Outputs are largely irrelevant or poor quality
Input Robustness	20%	Handles varied inputs robustly (different prompts/audio clips)	Handles common inputs well	Handles only the simplest inputs	Fails on basic inputs
Demo Polish	15%	Polished, engaging demo of both modalities	Clear demo with minor polish issues	Basic or rough demo	No usable demo presented
Tool/Model Explanation	10%	Clear explanation of models/tools used and their limitations	Adequate explanation	Minimal explanation	No explanation provided

Marks Evaluation:

Question No.	Pipeline Implementation (30%)	Output Quality & Relevance (25%)	Input Robustness (20%)	Demo Polish (15%)	Tool/Model Explanation (10%)	Total Marks (100)
Q1						
Q2						
					Overall Total (100)	